

Python Programming

We will be starting shortly

What song should I play today?



Quiz

Why don’t snakes like to weigh themselves?

[answer given in lecture]



Coursework 1

Well done on submitting on time!



Coursework 2

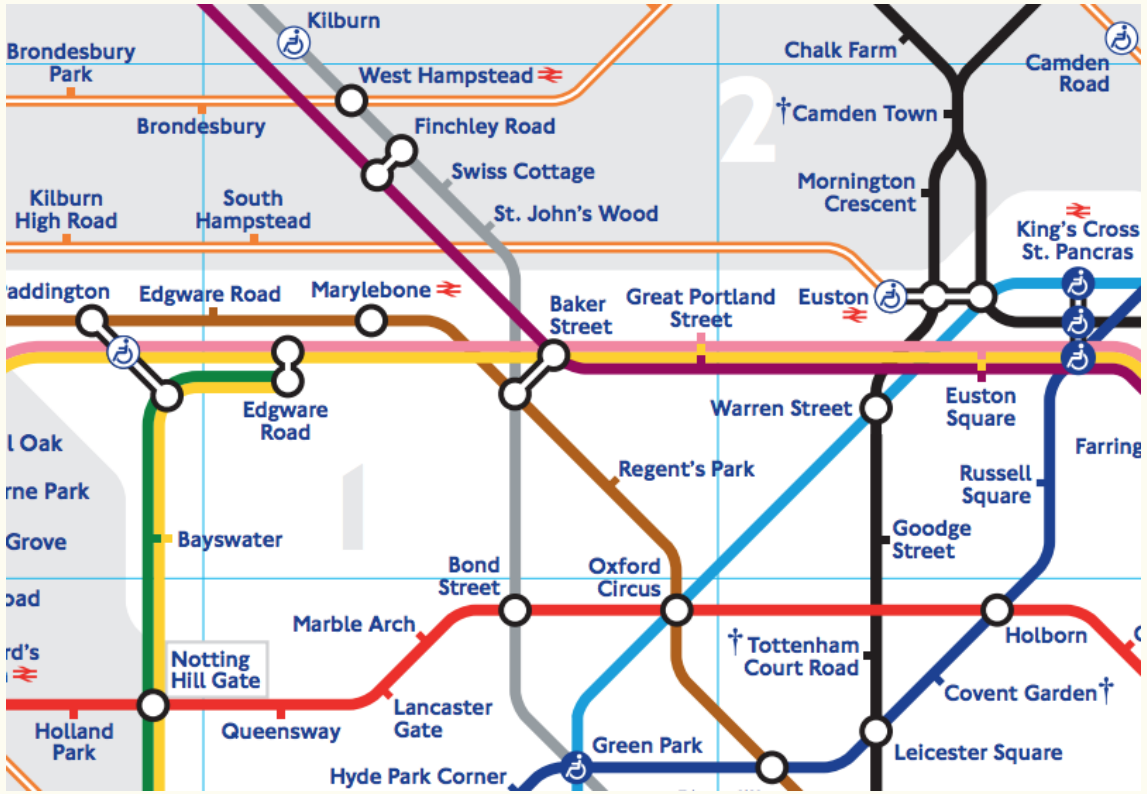
Deadline: **Mon 26th Oct 7pm GMT**



Clocks change on Sun 25th Oct!

Coursework 2

- 8% of final module grade





Coursework 2

- Aims:
  - Implement an existing algorithm
  - Python data structures (list, dict, set)
  - Read JSON files (easiest bit!)
  - Apply object-oriented programming
  - Modules & packages

- Use only Python Standard Library



Coursework 2

- Tasks (average 5-10 hours)
  - Complete the `TubeMap` class (read JSON file)
  - Complete the `NeighbourGraphBuilder` class
  - Complete the `PathFinder` class (Dijkstra's algorithm)

- GitLab Repo/LabTS
  - No more readme.pdf required!
  - Test your own code (no need to show us!)



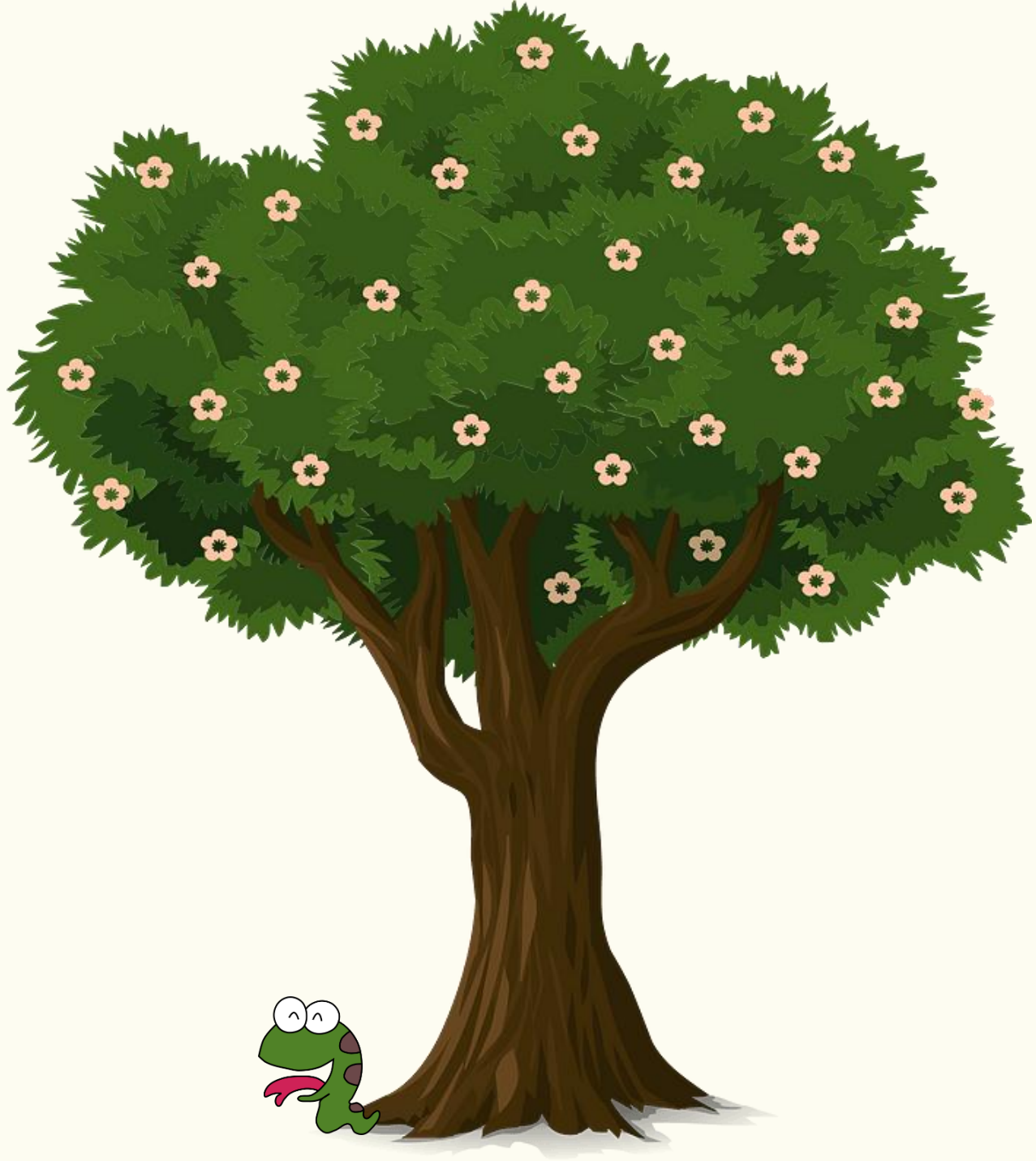
Trees

Pre-requisites: Recursion, OOP

Useful: Machine Learning, Logic & Symbolic AI,
Job interviews…



What is a Tree?







Node

Root node

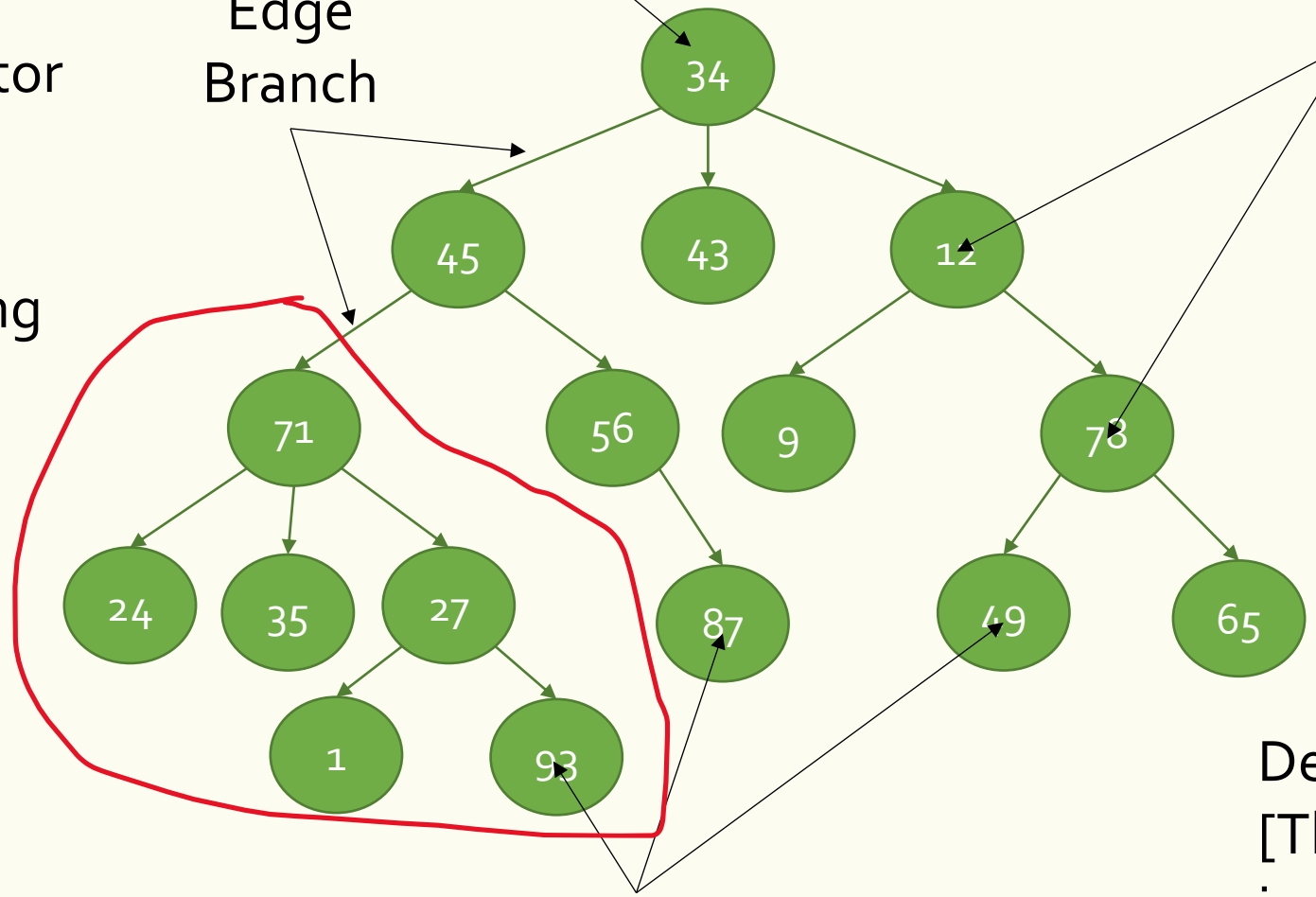
Edge
Branch

Ancestor

Parent

Child Sibling

Internal node
Non-terminal node
Intermediate node
Branch node



Height
[The **maximum** number of edges from root to leaf]

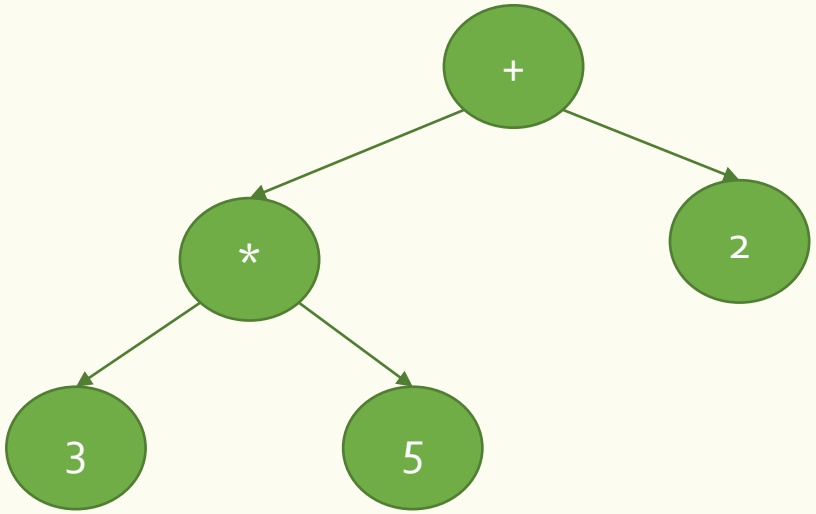
Subtree

Depth
[The number of edges in the path from a node to the root]

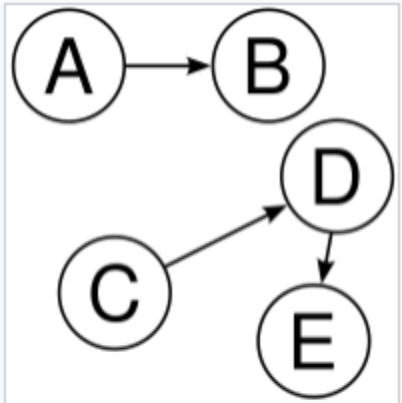
Leaf node
Terminal node



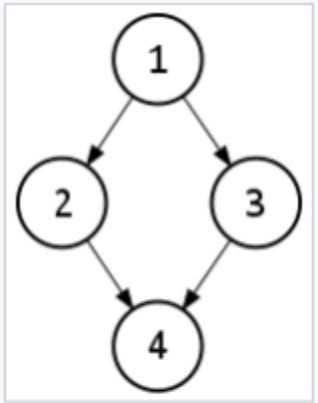
Application: Parsing



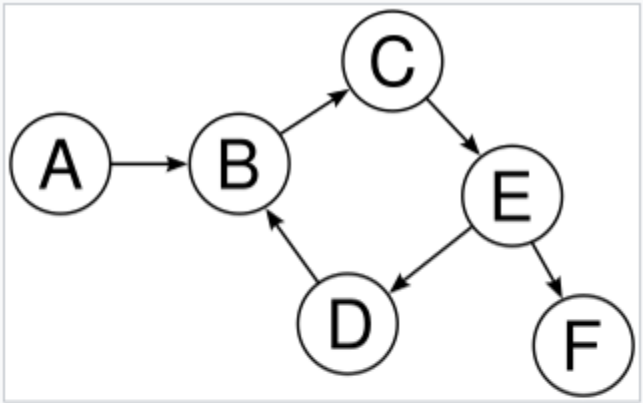
3 * 5 +2



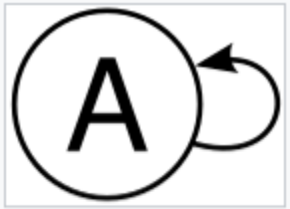
Not a tree: two non-connected parts, A→B and C→D→E. There is more than one root.



Not a tree: undirected cycle 1-2-4-3. 4 has more than one parent (inbound edge).



Not a tree: cycle B→C→E→D→B. B has more than one parent (inbound edge).



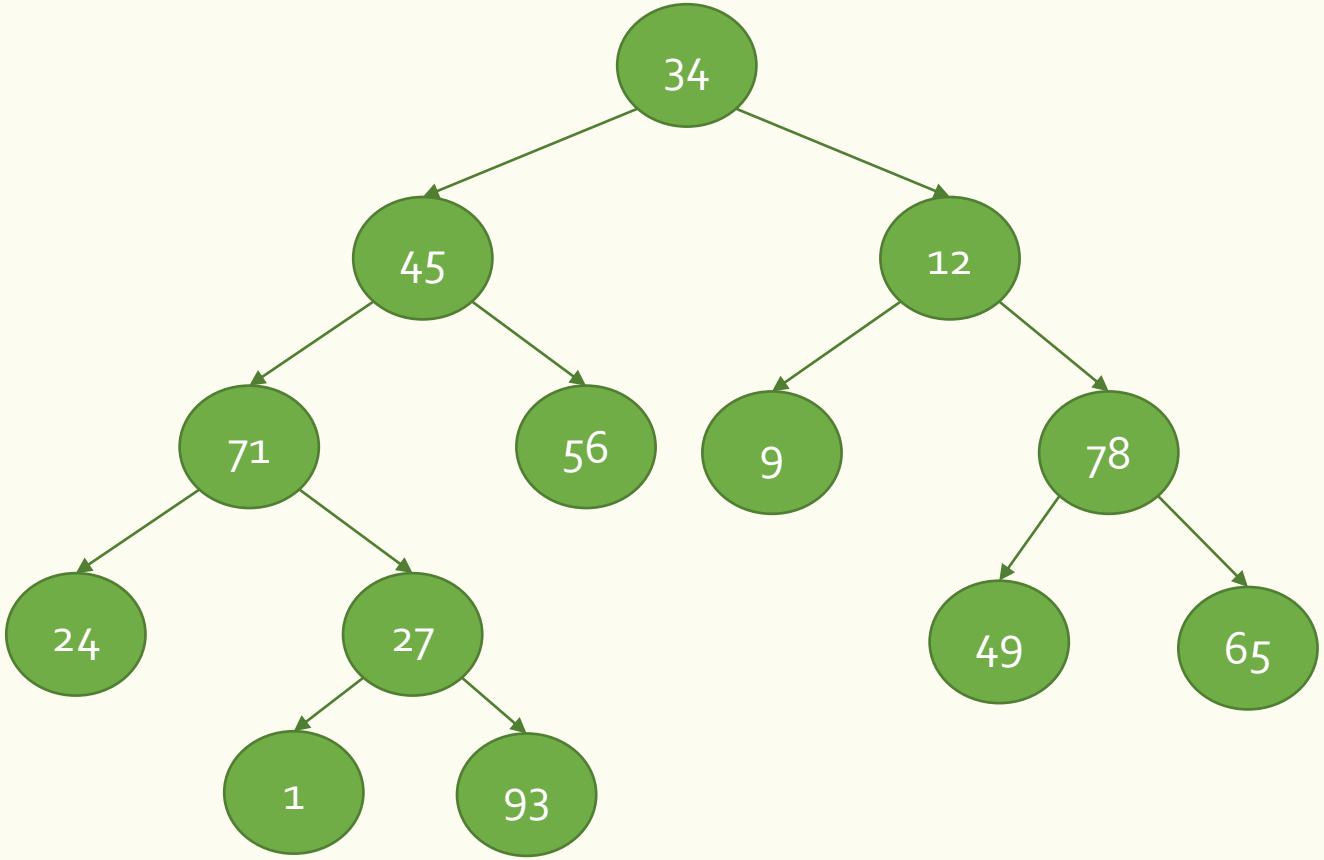
Not a tree: cycle A→A. A is the root but it also has a parent.



Each linear list is trivially a tree

Shamelessly copied from https://en.wikipedia.org/wiki/Tree_(data_structure)

Binary Tree





```
class BinaryTree:
    def __init__(self, value, left_child=None, right_child=None):
        self.value = value
        self.left_child = left_child
        self.right_child = right_child
```

```
class Tree:
    def __init__(self, value, children=None):
        self.value = value
        if children is None:
            self.children = []
        else:
            self.children = children
```

Binary Search (last week)

50	69	54	27	15	79	24	18	9	45	72	25



9	15	18	24	25	27	45	50	54	69	72	79

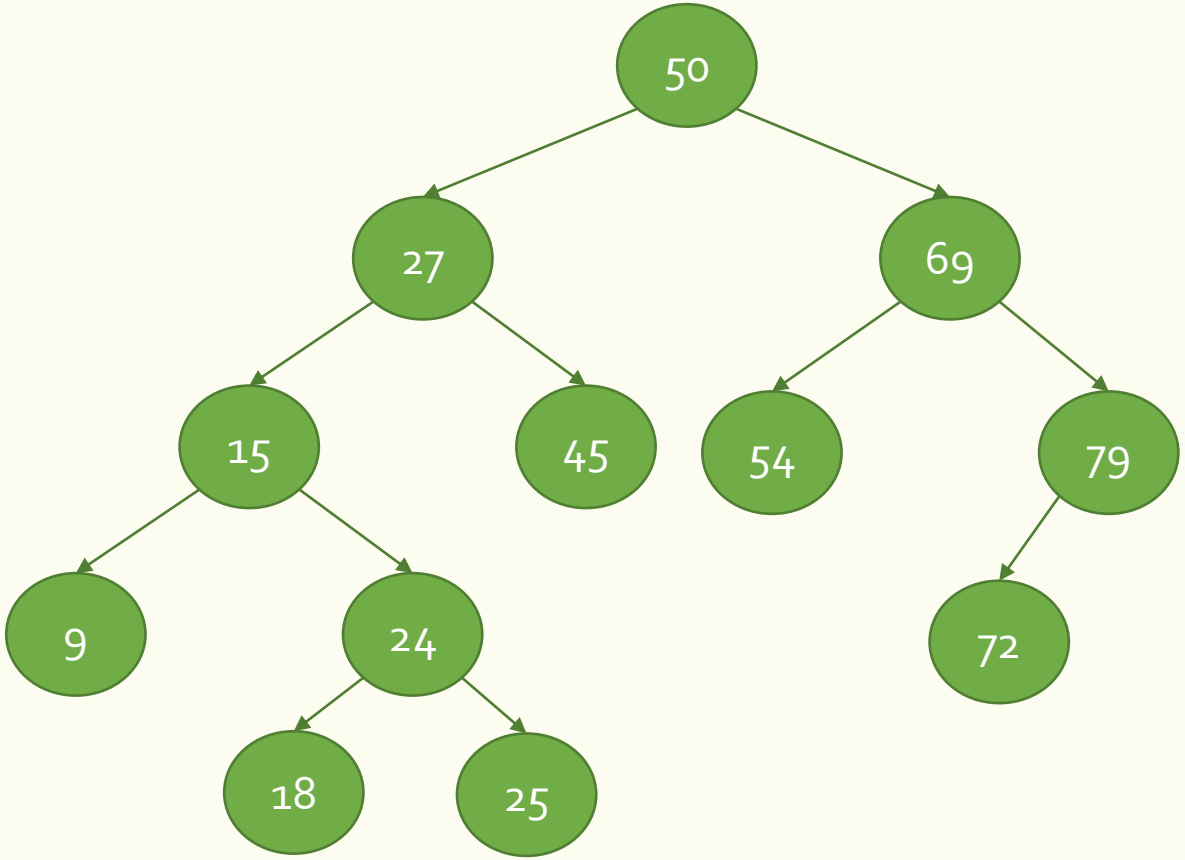


Binary Search implementation (recursive)

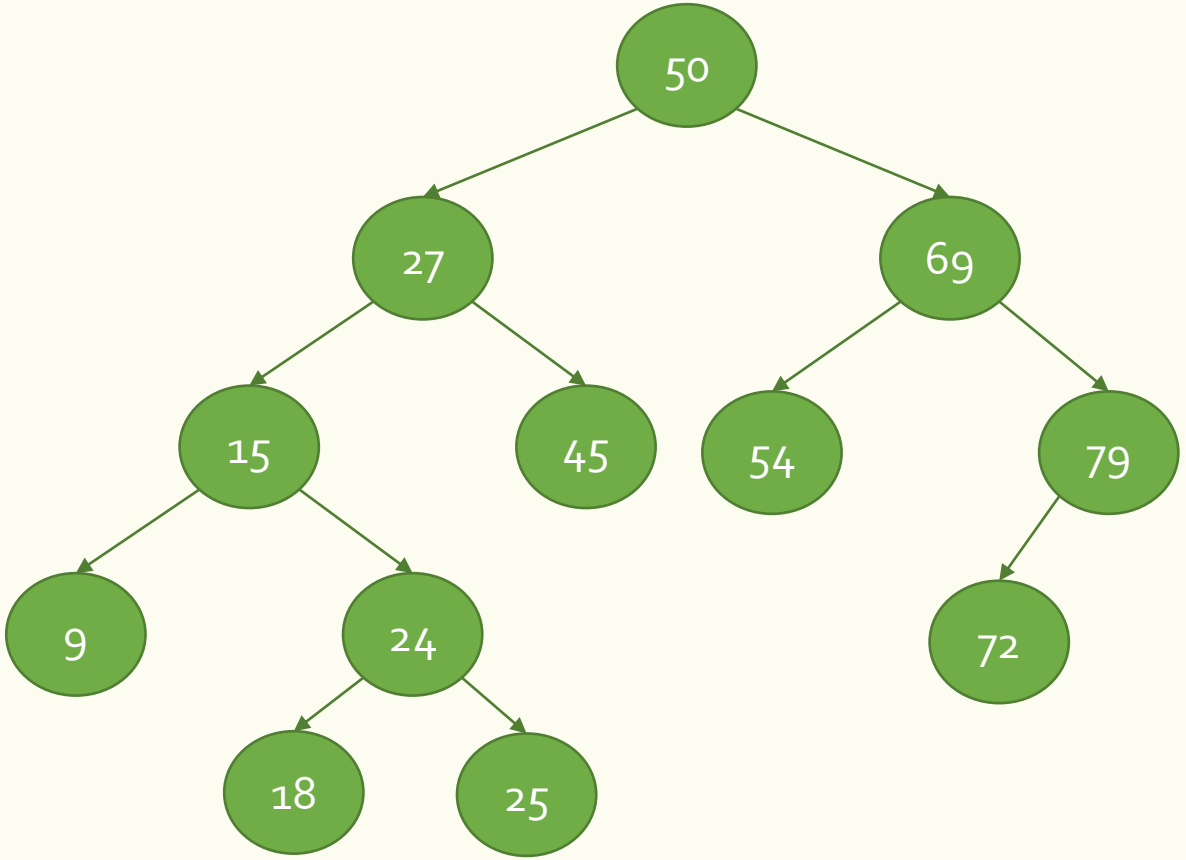
```
def binary_search(query, items):
    if len(items) == 1:
        return items[0] == query
    else:
        middle = round((len(items)-1)/2)
        if query == items[middle]:
            return True
        elif query < items[middle]:
            return binary_search(query, items[:middle])
        else:
            return binary_search(query, items[middle+1:])
```

Multiple searches?

Binary Search Tree



Binary Search Tree: Construction



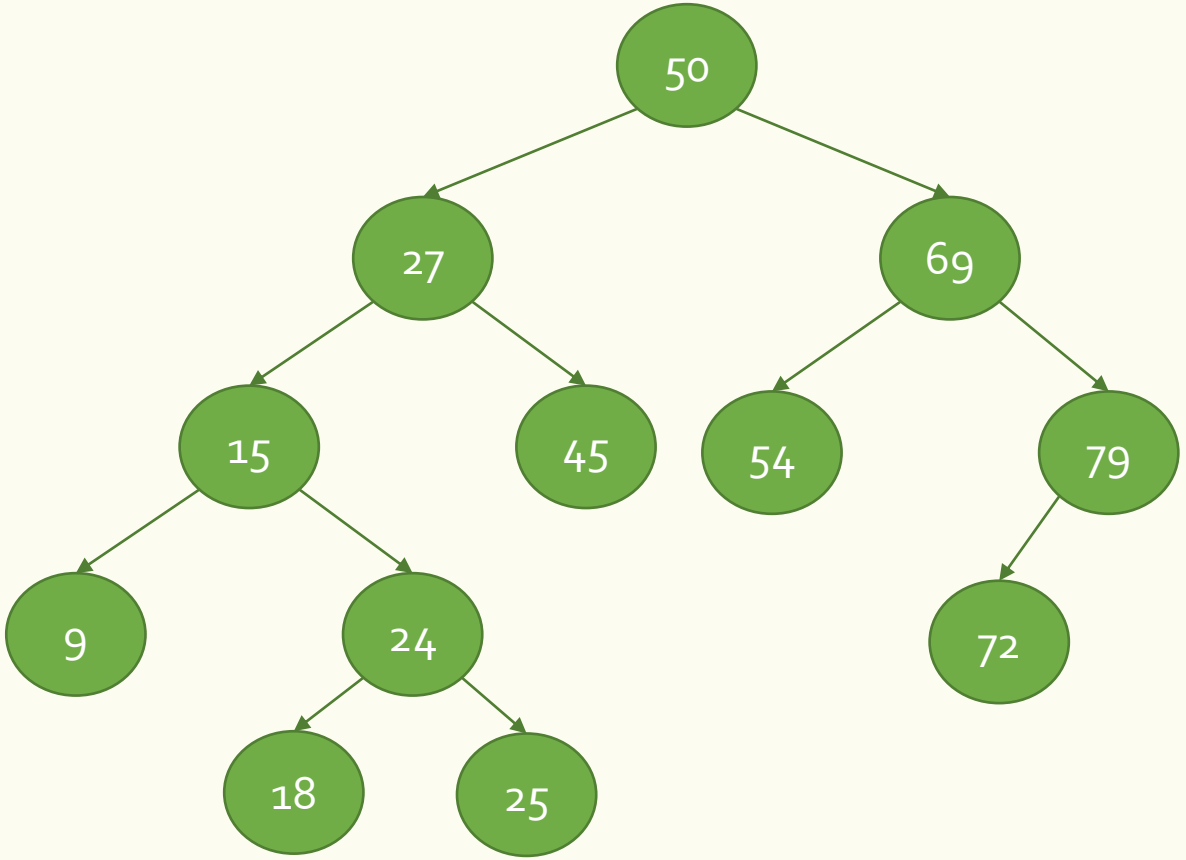
50	69	54	27	15	79	24	18	9	45	72	25



```
class BinarySearchTree:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left_child = left
        self.right_child = right
```

```
class BinarySearchTreeBuilder:
    def insert(self, value, node):
        if value ≤ node.value:
            if node.left_child is not None:
                self.insert(value, node.left_child)
            else:
                node.left_child = BinarySearchTree(value)
        else:
            if node.right_child is not None:
                self.insert(value, node.right_child)
            else:
                node.right_child = BinarySearchTree(value)
```

Binary Search Tree: Query

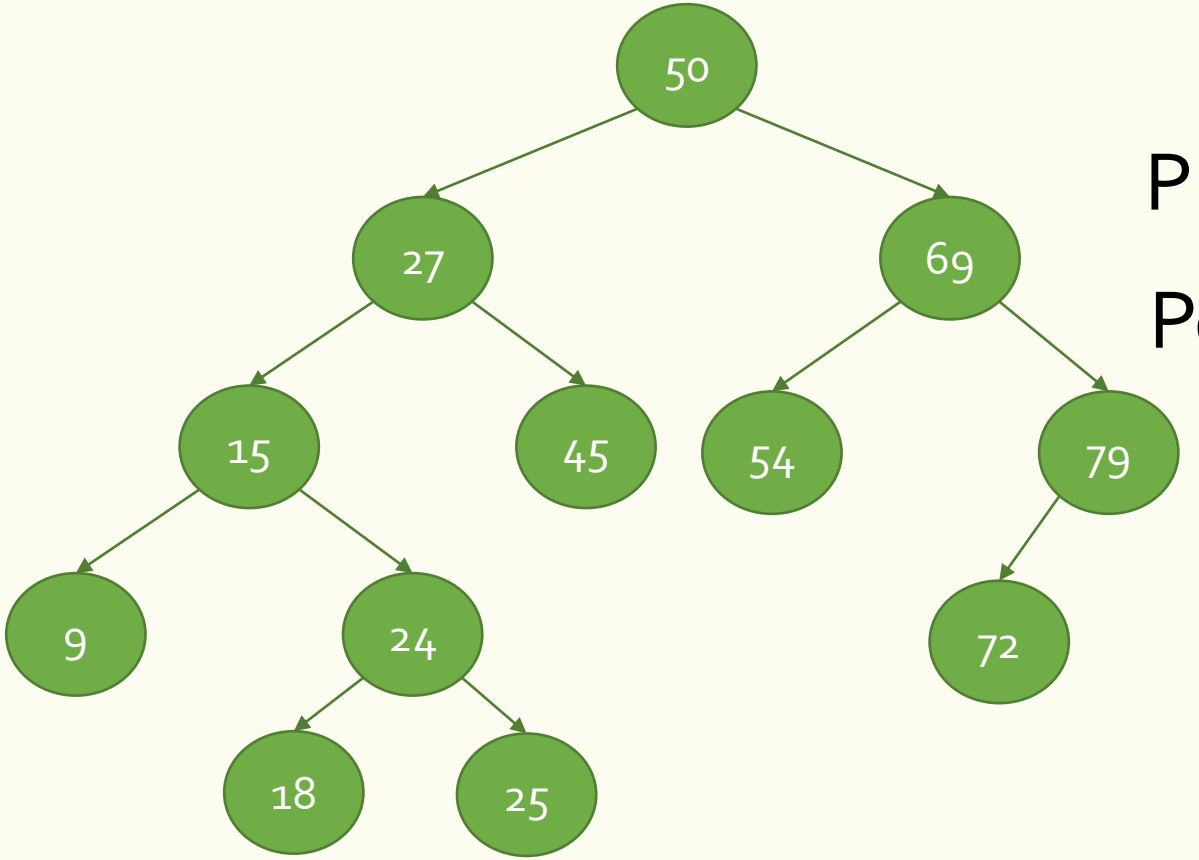




```
class BinarySearchTree:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left_child = left
        self.right_child = right
```

```
class BinarySearchTreeFinder:
    def find(self, query, node):
        if query == node.value:
            return True
        elif query < node.value:
            if node.left_child is not None:
                return self.find(query, node.left_child)
            else:
                return False
        else:
            if node.right_child is not None:
                return self.find(query, node.right_child)
            else:
                return False
```

Binary Search Tree: Traversal



Inorder [LVR] 1+2

Preorder [VLR] +12

Postorder [LRV] 12+



```
class BinarySearchTree:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left_child = left
        self.right_child = right
```

```
class BinarySearchTreeWalker:
    def inorder(self, node):
        if node.left_child is not None:
            self.inorder(node.left_child)

        print(node.value)

        if node.right_child is not None:
            self.inorder(node.right_child)
```

To accumulate the values (rather than printing), return a list and extend the list in the recursion



Summary

- Trees
  - Definitions
  - Binary trees
  - Binary search trees
    - Construction (Insertion)
    - Query
    - Traversal



This and next week's schedule

Thu 4-6pm	Fri 11am-1pm
LAB 202/206	LAB 202/206

Tue 2-3pm	Tue 3-4pm	Thu 4-6pm	Fri 11am-1pm
LAB 219	LECTURE 144	LAB 202/206	LAB 202/206

Next week's lecture topic: OOA&D

Any (anonymous) feedback for us?

- https://forms.gle/NbaFwtvT2xhWbeSW7